

Plaatscomplexiteit

1001WETALC

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Algoritme = TM dewelke stopt op elke input.

Hoe kunnen we nu de **plaats** meten die gebruikt wordt door een algoritme? Wat is plaats (space)?

Voor een **deterministische TM** met alfabet Σ :

- ▶ $space_M(w)$: Aantal tape cellen dat M "scant" op input $w \in \Sigma^*$.

Definitie

De "worst case" plaatscomplexiteit van een TM M is gedefinieerd als de functie $s_M : \mathbb{N} \rightarrow \mathbb{N}$:

$$s_M(n) = \max \{ space_M(w) \mid w \in \Sigma^* \text{ en } |w| = n \}.$$

Hier, $|w|$ is de lengte van de input.

Definitie

Een TM M gebruikt $\mathcal{O}(s(n))$ plaats als $s_M(n)$ van orde $\mathcal{O}(s(n))$ is.

Algoritme = TM dewelke stopt op elke input.

Hoe kunnen we nu de **plaats** meten die gebruikt wordt door een algoritme? Wat is plaats (space)?

Voor een **deterministische TM** met alfabet Σ :

- ▶ $space_M(w)$: Aantal tape cellen dat M "scant" op input $w \in \Sigma^*$.

Definitie

De "**worst case**" **plaatscomplexiteit** van een TM M is gedefinieerd als de functie $s_M : \mathbb{N} \rightarrow \mathbb{N}$:

$$s_M(n) = \max\{ space_M(w) \mid w \in \Sigma^* \text{ en } |w| = n \}.$$

Hier, $|w|$ is de lengte van de input.

Definitie

Een TM M **gebruikt** $\mathcal{O}(s(n))$ plaats als $s_M(n)$ van **orde** $\mathcal{O}(s(n))$ is.

We hebben een gelijkaardige definitie voor **niet-deterministische** TMs (NTMs):

Definitie

De “**worst case**” **plaatscomplexiteit** van een NTM M is gedefinieerd als de functie s_M die het **maximaal aantal cellen** meet dat M scant **in om het even welke tak** van de berekeningsboom van M op input woorden w van lengte $|w| = n$.

Herinner dat we veronderstellen dat M een beslisser is dus elke tak stopt op elke input.

Definitie

Een NTM M **gebruikt** $\mathcal{O}(s(n))$ plaats als $s_M(n)$ van **orde** $\mathcal{O}(s(n))$ is.

Net zoals voordien:

Definitie

Beschouw een functie $s : \mathbb{N} \rightarrow \mathbb{R}$. Dan,

$\text{DSPACE}(s(n)) = \{L \mid L \text{ is een taal die beslist wordt door een } \mathbf{deterministische} \text{ Turing machine die } \mathcal{O}(s(n)) \text{ plaats gebruikt}\}.$

$\text{NSPACE}(s(n)) = \{L \mid L \text{ is een taal die beslist wordt door een } \mathbf{niet-deterministische} \text{ Turing machine die } \mathcal{O}(s(n)) \text{ plaats gebruikt}\}.$

Voorbeeld

We tonen aan dat 3SAT in DSPACE(n) zit:

$M = \text{Op input } \langle \varphi \rangle$, met φ een 3SAT formule in veranderlijken $X = \{x_1, \dots, x_m\}$:

1. Voor elke truth assignment $\nu : X \rightarrow \{0, 1\}$ van de veranderlijken X in φ :
2. Test $\nu \models \varphi$.
3. Als een test positief is dan aanvaard, anders verwerp.

Cruciaal is dat we **plaats hergebruiken** voor de 2^m truth assignments van de veranderlijken!

Dit algoritme loopt in exponentiele tijd!

Voorbeeld

Zo ziet dit eruit op de tape (tijd loopt van boven naar beneden):

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	⊔	⊔	⊔	⊔	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	⊔	y	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	1	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	1	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Voorbeeld

Zo ziet dit eruit op de tape (tijd loopt van boven naar beneden):

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	⊔	⊔	⊔	⊔	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	⊔	y	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	1	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	1	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Voorbeeld

Zo ziet dit eruit op de tape (tijd loopt van boven naar beneden):

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	⊔	⊔	⊔	⊔	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	⊔	y	⊔	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	1	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	1	y	0	⊔	⊔
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Voorbeeld

Zo ziet dit eruit op de tape (tijd loopt van boven naar beneden):

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	⊔	⊔	⊔	⊔	⊔	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	⊔	y	⊔	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	0	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	1	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	1	y	0	⊔	⊔

Voorbeeld

Zo ziet dit eruit op de tape (tijd loopt van boven naar beneden):

⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	⊔	⊔	⊔	⊔	⊔	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	⊔	y	⊔	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	0	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	0	y	1	⊔	⊔
⊢	(	x	∨	¬	y	∨	x	)	∧	(	y	∨	x	∨	y	)	⊔	#	x	1	y	0	⊔	⊔

Wat meer details over TM M .

Voorbeeld

De TM $M = \text{Op input } \langle \varphi \rangle$ met φ een 3SAT formule bestaat uit:

- ▶ een TM M_{count} die “telt” van 0 tot $2^m - 1$ door binaire woorden van lengte m te genereren (één per één, hergebruik de plaats);
- ▶ een TM M_{eval} die φ evalueert op de truth assignment gerepresenteerd door het huidige binaire woord;
- ▶ aanvaard, als φ waar wordt. Zoniet, genereer hetvolgende woord (M_{count}) en begin opnieuw; en
- ▶ verwerp als geen enkele truth assignment φ waar maakt.

Elke van deze TM gebruikt $\mathcal{O}(m)$ plaats.

We zien dus een groot verschil tussen tijds- en plaatscomplexiteit!

Definitie

Een **eindige automaat** (Finite Automaton, FA) is gespecificeerd door $(Q, \Sigma, \delta, q_0, F)$ met

- ▶ Q eindige verzameling toestanden;
- ▶ Σ is het alfabet;
- ▶ q_0 is de begintoestand;
- ▶ $F \subseteq Q$ een eindige verzameling aanvaardingstoestanden; en
- ▶ $\delta : Q \times \Sigma \rightarrow Q$ is de transitiefunctie.

Een **niet-deterministische eindig automaat** (NFA) is van de vorm $(Q, \Sigma, \delta, q_0, F)$, net zoals voor FA, echter

- ▶ $\delta : Q \times \Sigma \rightarrow 2^Q$.

Dus meerdere toestanden mogelijk vanuit toestand en letter.

Definitie

Gegeven FA (NFA) M , de (**reguliere**) taal $L(M)$ **aanvaard** door M bestaat uit alle woorden in $w \in \Sigma^*$ die aanvaard worden door M (ttz. $\delta(q_0, w) \in F$).

Definitie

Het **universality** probleem voor een NFA A is

$$\text{ALL}_{\text{NFA}} = \{ \langle A \rangle \mid A \text{ is een NFA en } L(A) = \Sigma^* \}$$

We kunnen, equivalent, bekijken of de **complement** automaat $\bar{A}$ niet leeg is.

$$w \in L(\bar{A}) \Leftrightarrow w \notin L(A).$$

We willen dus testen of de complement automaat een niet-ledige taal aanvaardt.

Beschouw NFA $N = (Q, \Sigma, \delta, q_0, F)$.

Naïeve methode: Herleid naar het **testen of een niet-lege taal aanvaard wordt door een FA**.

1. **Transformeer** N naar een DFA $D = (Q', \Sigma', \delta', q'_0, F')$ ($L(N) = L(D)$);
2. **Verwissel** aanvaardingstoestanden met alle andere (niet aanvaardingstoestanden): $D' = (Q', \Sigma', \delta', q'_0, Q' \setminus F')$ ($L(D') = L(\bar{N})$);
3. **Bereken bereikbare toestanden** $\text{reach}(q'_0)$ in D' vanuit q'_0 . Als $\text{reach}(q'_0) \cap F' = \emptyset$ dan is $L(\bar{N})$ niet leeg (en **verwerp**), anders is $L(\bar{N})$ leeg (en **aanvaard**).

Herinner de **subset constructie**:

- $D = (Q' = 2^Q, \Sigma' = \Sigma, \delta', q'_0 = \{q_0\}, F')$ met $F' = \{q' \subseteq Q \mid q' \cap F \neq \emptyset\}$ en

$$\delta'(q', a) = \bigcup_{q \in q'} \delta(q, a).$$

Dit zorgt voor een **exponentiële blow-up** in het aantal toestanden!

Het berekenen van $\text{reach}(q'_0)$ vraagt polynomiaal veel tijd in het aantal toestanden.

Dit algoritme vraagt dus **exponentieel** veel tijd en ruimte

Beschouw NFA $N = (Q, \Sigma, \delta, q_0, F)$.

Naïeve methode: Herleid naar het **testen of een niet-lege taal aanvaard wordt door een FA**.

1. **Transformeer** N naar een DFA $D = (Q', \Sigma', \delta', q'_0, F')$ ($L(N) = L(D)$);
2. **Verwissel** aanvaardingstoestanden met alle andere (niet aanvaardingstoestanden): $D' = (Q', \Sigma', \delta', q'_0, Q' \setminus F')$ ($L(D') = L(\bar{N})$);
3. **Bereken bereikbare toestanden** $\text{reach}(q'_0)$ in D' vanuit q'_0 . Als $\text{reach}(q'_0) \cap F' = \emptyset$ dan is $L(\bar{N})$ niet leeg (en **verwerp**), anders is $L(\bar{N})$ leeg (en **aanvaard**).

Herinner de **subset constructie**:

- $D = (Q' = 2^Q, \Sigma' = \Sigma, \delta', q'_0 = \{q_0\}, F')$ met $F' = \{q' \subseteq Q \mid q' \cap F \neq \emptyset\}$ en

$$\delta'(q', a) = \bigcup_{q \in q'} \delta(q, a).$$

Dit zorgt voor een **exponentiële blow-up** in het aantal toestanden!

Het berekenen van $\text{reach}(q'_0)$ vraagt polynomiaal veel tijd in het aantal toestanden.

Dit algoritme vraagt dus **exponentieel** veel tijd en ruimte

Tweede poging: vermijd de constructie van de complement automaat.

$M = \text{Op input } \langle A \rangle$, voor NFA A :

1. **Markeer de starttoestand** q_0 van A ;
2. Kies op een **niet-deterministisch** manier een **woord** w **van lengte** 2^q , met q het aantal toestanden in A .
3. **Simuleer** $\delta(q_0, w)$ door markering te plaatsen op de toestanden telkens een symbool van w gelezen wordt.
4. Wanneer er ooit een aanvaardingstoestand gemarkeerd is (en dus een toestand $q \in 2^Q$ bereikt werd met $q \cap F \neq \emptyset$ and dus $q \in F'$), verwerp. Anders, aanvaard (en dus is w een woord dat niet aanvaard wordt door A .)

Dit is duidelijk een correct algoritme:

- ▶ Als er een woord $w \in L(\bar{A})$ bestaat, bestaat er zeker een van lengte 2^q .
- ▶ Voor woorden van langere lengte moet er een herhaling zijn en kan het woord worden ingekort.

We kiezen een woord van lengte 2^q : dit is in $NSPACE(2^q)$.

Kunnen we beter?

Tweede poging: vermijd de constructie van de complement automaat.

$M = \text{Op input } \langle A \rangle$, voor NFA A :

1. **Markeer de starttoestand** q_0 van A ;
2. Kies op een **niet-deterministisch** manier een **woord** w **van lengte** 2^q , met q het aantal toestanden in A .
3. **Simuleer** $\delta(q_0, w)$ door markering te plaatsen op de toestanden telkens een symbool van w gelezen wordt.
4. Wanneer er ooit een aanvaardingstoestand gemarkeerd is (en dus een toestand $q \in 2^Q$ bereikt werd met $q \cap F \neq \emptyset$ and dus $q \in F'$), verwerp. Anders, aanvaard (en dus is w een woord dat niet aanvaard wordt door A .)

Dit is duidelijk een correct algoritme:

- ▶ Als er een woord $w \in L(\bar{A})$ bestaat, bestaat er zeker een van lengte 2^q .
- ▶ Voor woorden van langere lengte moet er een herhaling zijn en kan het woord worden ingekort.

We kiezen een woord van lengte 2^q : dit is in $NSPACE(2^q)$.

Kunnen we beter?

Tweede poging: vermijd de constructie van de complement automaat.

$M = \text{Op input } \langle A \rangle$, voor NFA A :

1. **Markeer de starttoestand** q_0 van A ;
2. Kies op een **niet-deterministisch** manier een **woord** w **van lengte** 2^q , met q het aantal toestanden in A .
3. **Simuleer** $\delta(q_0, w)$ door markering te plaatsen op de toestanden telkens een symbool van w gelezen wordt.
4. Wanneer er ooit een aanvaardingstoestand gemarkeerd is (en dus een toestand $q \in 2^Q$ bereikt werd met $q \cap F \neq \emptyset$ and dus $q \in F'$), verwerp. Anders, aanvaard (en dus is w een woord dat niet aanvaard wordt door A .)

Dit is duidelijk een correct algoritme:

- ▶ Als er een woord $w \in L(\bar{A})$ bestaat, bestaat er zeker een van lengte 2^q .
- ▶ Voor woorden van langere lengte moet er een herhaling zijn en kan het woord worden ingekort.

We kiezen een woord van lengte 2^q : dit is in $NSPACE(2^q)$.

Kunnen we beter?

Derde poging:

$M = \text{Op input } \langle A \rangle$, voor een NFA A :

1. **Markeer de starttoestand** q_0 van A ;
2. Herhaal het volgende 2^q keer:
3. Kies op een **niet-deterministische** manier een **letter** in Σ en verander de markeringen op toestanden om zo de transities van A te **simuleren**.
4. Wanneer er ooit een aanvaardingstoestand gemarkeerd is, verwerp. Anders, aanvaard.

Dit is een algoritme voor $\overline{ALL}_{NFA}$ in $NSPACE(n)$.

We komen later terug op de plaatscomplexiteit van ALL_{NFA} .

De volgende klassen zijn belangrijk:

Definitie

$$\text{PSPACE} = \bigcup_k \text{DSPACE}(n^k).$$

$$\text{NPSPACE} = \bigcup_k \text{NSPACE}(n^k).$$

Voorbeeld

We hebben gezien dat 3SAT in PSPACE zit en $\overline{\text{ALL}}_{\text{NFA}}$ in NPSPACE.

Definitie

Een **gekwantificeerde propositionele formule** (**QBF**, quantified Boolean Formula) is een formule van de volgende vorm:

$$Q_1 x_1 Q_2 x_2 \dots Q_k x_k \varphi$$

met daarin

- ▶ $Q_i \in \{\forall, \exists\}$ de **universele** en **existentiële kwantoren**, en
- ▶ φ is een propositional formule over de veranderlijken $x_1, \dots, x_k$.

De semantiek is zoals verwacht.

Voorbeeld

Welke formule is **waar**?

- ▶ $\exists y \forall x (x \vee y) \wedge (\neg x \vee \neg y)$
- ▶ $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

Voorbeeld

y	x	φ
0	0	0
0	1	1
1	0	1
1	1	0

- ▶ $\exists y \forall x (x \vee y) \wedge (\neg x \vee \neg y)$. Niet waar.
 - ▶ $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$. Waar.
- ▶ In tegenstelling tot propositionele formules **vraagt dit meer werk** om de waarheid van QBFs te achterhalen!

Een gekwantificeerd propositionele formule wordt “**volledig**” genoemd als elke veranderlijke gekwantificeerd is.

Definitie

TQBF = $\{\langle \phi \rangle \mid \phi \text{ is een volledige gekwantificeerde}$
propositionele formule die waar is}.

Stelling

TQBF is PSPACE-compleet.

Propositionele formules, ditmaal gekwantificeerde formules, zorgen dus opnieuw voor een karakterisatie van een complexiteitsklasse, ditmaal PSPACE.

We gaan PSPACE-compleetheid later definiëren.

We kijken alvast eerst na of TQBF in PSPACE zit.

Bewijs: We moeten een TM maken die in $\mathcal{O}(n^k)$ plaats loopt.

$M =$ Op input $\langle \phi \rangle$, met ϕ een volledige gekwantificeerde propositionele formule:

1. Als er geen kwantoren meer zijn, dan hebben we een propositionele formule waarin alle veranderlijken waarden hebben (0 of 1). Aanvaard als deze waar is, verwerp anders.
2. Als $\phi = \forall x \varphi$ dan roepen we M **recursief** aan op φ :

► Eenmaal met $x = 1$, eenmaal met $x = 0$.

Aanvaard als M op beide inputs aanvaardt, anders verwerp.

3. Als $\phi = \exists x \varphi$ dan roepen we M **recursief** aan op φ .

► Eenmaal met $x = 1$, eenmaal met $x = 0$.

Aanvaard als M op één van de twee tests aanvaardt, anders verwerp.

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.



$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.



$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$		0
----------	--	---

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	1
----------	---	---

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

true

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	
----------	---	--

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

true

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	1
----------	---	---

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

true

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	1
----------	---	---

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$

true

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$

true

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	
----------	---	--

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$

call $y = 0$

$\exists x (x \vee 0) \wedge (\neg x \vee 1)$ **true**

call $x = 0$

$(0 \vee 0) \wedge (1 \vee 1)$ false

call $x = 1$

$(1 \vee 0) \wedge (0 \vee 1)$ true

call $y = 1$

$\exists x (x \vee 1) \wedge (\neg x \vee 0)$ **true**

call $x = 0$

$(0 \vee 1) \wedge (1 \vee 0)$ true

call $x = 1$

$(1 \vee 1) \wedge (0 \vee 0)$ false

Voorbeeld

Beschouw: $\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$.

$\vdash$	1	
----------	---	--

$\forall y \exists x (x \vee y) \wedge (\neg x \vee \neg y)$	true		
call $y = 0$	$\exists x (x \vee 0) \wedge (\neg x \vee 1)$	true	
	call $x = 0$	$(0 \vee 0) \wedge (1 \vee 1)$	false
	call $x = 1$	$(1 \vee 0) \wedge (0 \vee 1)$	true
call $y = 1$	$\exists x (x \vee 1) \wedge (\neg x \vee 0)$	true	
	call $x = 0$	$(0 \vee 1) \wedge (1 \vee 0)$	true
	call $x = 1$	$(1 \vee 1) \wedge (0 \vee 0)$	false

In het algemeen:

- ▶ De **diepte van de recursie** is k (het aantal veranderlijken).
- ▶ Elke recursieve aanroep vraagt slechts om een waarde 0 (false) of 1 (true) **extra te bewaren**.
- ▶ We **hergebruiken plaats**.

We hebben dus $\mathcal{O}(k)$ plaats nodig om TQBF te beslissen en dus TQBF zit in PSPACE.

De volgende relaties gelden (bijna vanzelfsprekend):

- ▶ $PSPACE \subseteq NPSPACE$, omdat elk TM een NTM is.
- ▶ $PTIME \subseteq PSPACE$, omdat we in polynomiale tijd niet meer dan een polynomiaal aantal cellen op de tape kunnen scannen.
- ▶ Om dezelfde reden: $NP \subseteq NPSPACE$.

Is $PSPACE=NPSPACE$ ook een miljoen dollar waard?

De volgende relaties gelden (bijna vanzelfsprekend):

- ▶ $PSPACE \subseteq NPSPACE$, omdat elk TM een NTM is.
- ▶ $PTIME \subseteq PSPACE$, omdat we in polynomiale tijd niet meer dan een polynomiaal aantal cellen op de tape kunnen scannen.
- ▶ Om dezelfde reden: $NP \subseteq NPSPACE$.

Is $PSPACE=NPSPACE$ ook een miljoen dollar waard? Nee!

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Stelling (Savitch)

Als $s(n) \geq n$, dan

$$NSPACE(s(n)) \subseteq DSPACE((s(n))^2).$$

en dus

Corollary

$PSPACE = NPSPACE$.

Bewijs: We hebben al dat $PSPACE \subseteq NPSPACE$. Daarenboven,

$$NPSPACE = \bigcup_k NSPACE(n^k) \subseteq \bigcup_k DSPACE(n^{2k}) \subseteq PSPACE,$$

waar de tweede inclusie geldt omwille van de Stelling van Savitch. □

We hebben dus:

$$PTIME \subseteq NP \subseteq PSPACE = NPSPACE.$$

Het is niet geweten of deze inclusies strikt zijn!

Stelling (Savitch)

Als $s(n) \geq n$, dan

$$NSPACE(s(n)) \subseteq DSPACE((s(n))^2).$$

en dus

Corollary

$PSPACE = NPSPACE$.

Bewijs: We hebben al dat $PSPACE \subseteq NPSPACE$. Daarenboven,

$$NPSPACE = \bigcup_k NSPACE(n^k) \subseteq \bigcup_k DSPACE(n^{2k}) \subseteq PSPACE,$$

waar de tweede inclusie geldt omwille van de Stelling van Savitch. □

We hebben dus:

$$PTIME \subseteq NP \subseteq PSPACE = NPSPACE.$$

Het is niet geweten of deze inclusies strikt zijn!

Stelling (Savitch)

Als $s(n) \geq n$, dan

$$NSPACE(s(n)) \subseteq DSPACE((s(n))^2).$$

en dus

Corollary

$PSPACE = NPSPACE$.

Bewijs: We hebben al dat $PSPACE \subseteq NPSPACE$. Daarenboven,

$$NPSPACE = \bigcup_k NSPACE(n^k) \subseteq \bigcup_k DSPACE(n^{2k}) \subseteq PSPACE,$$

waar de tweede inclusie geldt omwille van de Stelling van Savitch. □

We hebben dus:

$$PTIME \subseteq NP \subseteq PSPACE = NPSPACE.$$

Het is niet geweten of deze inclusies strikt zijn!

We hebben eerder gezien dat $\overline{ALL}_{NFA}$ in $NSPACE(n)$ zit.

Nu weten we dat $NSPACE(n) \subseteq DSPACE(n^2)$ en dus $\overline{ALL}_{NFA} \in DSPACE(n^2)$.

Aangezien deterministische complexiteitsklassen **gesloten zijn onder complementatie** is dus ALL_{NFA} in $DSPACE(n^2)$.

Dus ALL_{NFA} is in $PSPACE$.

Stelling (Savitch)

Als $s(n) \geq n^a$, dan

$$NSPACE(s(n)) \subseteq DSPACE((s(n))^2).$$

^aWe veronderstellen dat de input op de tape gezet kan worden.

Dit betekent dat we een $\mathcal{O}(s(n))$ -plaats NTM N **deterministisch moeten simuleren** door een $\mathcal{O}((s(n))^2)$ -plaats TM M .

Hoe?

- ▶ Beschouw configuraties van N op input woord w van lengte n .
- ▶ Representeer de berekening van N als een gerichte graaf (**configuratiegraaf**).
- ▶ Test of een aanvaardingsconfiguratie (een knoop in de graaf) **bereikt** kan worden vanuit de initiële configuratie (ook een knoop in de graaf).

Stelling (Savitch)

Als $s(n) \geq n^a$, dan

$$NSPACE(s(n)) \subseteq DSPACE((s(n))^2).$$

^aWe veronderstellen dat de input op de tape gezet kan worden.

Dit betekent dat we een $\mathcal{O}(s(n))$ -plaats NTM N **deterministisch moeten simuleren** door een $\mathcal{O}((s(n))^2)$ -plaats TM M .

Hoe?

- ▶ Beschouw configuraties van N op input woord w van lengte n .
- ▶ Representeer de berekening van N als een gerichte graaf (**configuratiegraaf**).
- ▶ Test of een aanvaardingsconfiguratie (een knoop in de graaf) **bereikt** kan worden vanuit de initiële configuratie (ook een knoop in de graaf).

Eerste vraag: Hoeveel configuraties zijn er voor een $\mathcal{O}(s(n))$ -plaats NTM N op input een woord w van lengte n ?

We schatten eerst het **aantal configuraties** af.

- ▶ Een configuratie: $\vdash uqv$ met $u, v \in \Gamma^*$ and $q \in Q$.
- ▶ Woorden uv zijn maximaal $ds(n)$ lang voor een constante d .
- ▶ Er zijn $|Q|$ mogelijke toestanden, $|\Gamma|$ mogelijke tape-symbolen
- ▶ De lees/schrijfkop kan op maximaal $ds(n)$ posities staan.

Dus, we hebben maximaal

$$d|Q|s(n)|\Gamma|^{ds(n)} = 2^{\log(d)+\log(|Q|)+\log(s(n))+\log(|\Gamma|)ds(n)} = 2^{\mathcal{O}(s(n))}$$

configuraties.

Definitie

Configuratiegraaf $G(N, w)$ van NTM N op input woord w :

- ▶ Knopen: configuraties c van N op w
- ▶ Bogen: Als c_2 bereikbaar is vanuit c_1 dmv transities in N .

Het is duidelijk dat we deze graaf **niet kunnen opslaan** in $\mathcal{O}((s(n))^2)$ plaats.

Tweede vraag: Hoeveel stappen kan een $\mathcal{O}(s(n))$ -plaats NTM N op input een woord w van lengte n doen?

- ▶ We hebben maximaal $2^{es(n)}$ configuraties, voor een constante e .
- ▶ Dus N kan maximaal $2^{es(n)}$ stappen zetten (ander is er een herhaling van een configuratie en zou N blijven lopen).

We kunnen ons dus **beperken tot paden in de configuratiegraaf $G(N, w)$ van lengte $2^{es(n)}$** .

Procedure **IsBereikbaar**(c_1, c_2, t):

- ▶ **Input:** configuraties c_1 and c_2 van N op w , t is maximaal aantal stappen dat N mag doen.
- ▶ **Output:** Aanvaard als c_2 bereikbaar is vanuit c_1 in een tak in de berekeningsboom van N op w in maximaal t stappen. Verwerp, anders.

We veronderstellen dat N een unieke aanvaardingstoestand c_{accept} heeft.

- ▶ Dit kan steeds door een unieke nieuwe toestand toe te voegen.

Hoe helpt dit ons?

Beschouw een deterministische TM M die op input w het resultaat teruggeeft van

$$\text{IsBereikbaar}(c_{ini}, c_{accept}, 2^{e s(n)}),$$

waar e een constante is zodat $2^{e s(n)}$ een bovengrens is op het aantal stappen dat N doet.

We **veronderstellen even** dat $s(n)$ berekenbaar is $\mathcal{O}((s(n))^2)$ plaats. (omdat M $s(n)$ nodig heeft).

Procedure **IsBereikbaar**(c_1, c_2, t):

- ▶ **Input:** configuraties c_1 and c_2 van N op w , t is maximaal aantal stappen dat N mag doen.
- ▶ **Output:** Aanvaard als c_2 bereikbaar is vanuit c_1 in een tak in de berekeningsboom van N op w in maximaal t stappen. Verwerp, anders.

We veronderstellen dat N een unieke aanvaardingstoestand c_{accept} heeft.

- ▶ Dit kan steeds door een unieke nieuwe toestand toe te voegen.

Hoe helpt dit ons?

Beschouw een deterministische TM M die op input w het resultaat teruggeeft van

$$\text{IsBereikbaar}(c_{ini}, c_{accept}, 2^{e s(n)}),$$

waar e een constante is zodat $2^{e s(n)}$ een bovengrens is op het aantal stappen dat N doet.

We **veronderstellen even** dat $s(n)$ berekenbaar is $\mathcal{O}((s(n))^2)$ plaats. (omdat M $s(n)$ nodig heeft).

Als de procedure **IsBereikbaar**($C_{ini}, C_{accept}, 2^{es(n)}$) het juiste resultaat teruggeeft dan aanvaard M dezelfde taal als N :

Input w wordt aanvaard door N als en slechts als er een pad in de configuratiegraaf $G(N, w)$ bestaat tussen de initiële en aanvaardingstoestand.

We kunnen dus onze aandacht verschuiven naar de procedure **IsBereikbaar**.

Idee: Elk pad van lengte 2^i is een samenstelling van 2 paden van lengte 2^{i-1} .

IsBereikbaar(c_1, c_2, t)

1. Als $t = 1$, check ofwel $c_1 = c_2$, ofwel dat c_2 bereikbaar is vanuit c_1 in een stap dmv transitierelatie van N . Aanvaard als één van deze twee testen slaagt, zoniet verwerp.
2. Als $t > 1$ dan voor **alle mogelijke configuraties** c_m van N van lengte $ds(n) + 2$ (lengte woord plus toestand and $\vdash$):
 3. Voer IsBereikbaar($c_1, c_m, \frac{t}{2}$) uit
 4. Voer IsBereikbaar($c_m, c_2, \frac{t}{2}$) uit
 5. Als beiden aanvaarden, aanvaard.
6. Als nog niet aanvaard, verwerp.

Heeft IsBereikbaar inderdaad slechts $\mathcal{O}((s(n))^2)$ plaats nodig?

$\vdash (c_{ini} c_{accept} 0)$	Initiële aanroep $\text{IsBereikbaar}(c_{ini}, c_{accept}, t/2^0 = t)?$
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1)$	$\text{IsBereikbaar}(c_{ini}, c_m^1, t/2)?$
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1) (c_{ini} c_m^1 2)$	$\text{IsBereikbaar}(c_{ini}, c_m^1, t/4)$: Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1) (c_{ini} c_m^2 2)$	$\text{IsBereikbaar}(c_{ini}, c_m^2, t/4)$: Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1) (c_m^2 c_m^1 2)$	$\text{IsBereikbaar}(c_m^2, c_m^1, t/4)$: Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1) (c_{ini} c_m^3 2)$	$\text{IsBereikbaar}(c_{ini}, c_m^3, t/4)$: Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1) (c_m^3 c_m^1 2)$	$\text{IsBereikbaar}(c_m^3 c_m^1, t/4)$: Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^1 1)$	$\text{IsBereikbaar}(c_{ini}, c_m^1, t/2)$: Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^2 1)$	$\text{IsBereikbaar}(c_{ini}, c_m^2, t/2)?$
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^2 1) (c_{ini} c_m^1 2)$	$\text{IsBereikbaar}(c_{ini}, c_m^1, t/4)$: Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^2 1) (c_{ini} c_m^2 2)$	$\text{IsBereikbaar}(c_{ini}, c_m^2, t/4)$: Aanvaard

$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^2 1) (c_m^2 c_m^2 2) \}$	IsBereikbaar($c_m^2, c_m^2, t/4$): Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_{ini} c_m^2 1) \}$	IsBereikbaar($c_{ini}, c_m^2, t/2$): Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_m^2 c_{accept} 1) \}$	IsBereikbaar($c_m^2, c_{accept}, t/2$): ?
$\vdash (c_{ini} c_{accept} 0) (c_m^2 c_{accept} 1) (c_m^2 c_m^1 2) \}$	IsBereikbaar($c_m^2, c_m^1, t/4$): Verwerp
$\vdash (c_{ini} c_{accept} 0) (c_m^2 c_{accept} 1) (c_m^2 c_m^2 2) \}$	IsBereikbaar($c_m^2, c_m^2, t/4$): Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_m^2 c_{accept} 1) (c_m^2 c_{accept} 2) \}$	IsBereikbaar($c_m^2, c_{accept}, t/4$): Aanvaard
$\vdash (c_{ini} c_{accept} 0) (c_m^2 c_{accept} 1) \}$	IsBereikbaar($c_m^2, c_{accept}, t/2$): Aanvaard
$\vdash (c_{ini} c_{accept} 0) \}$	IsBereikbaar($c_{ini}, c_{accept}, t/2^0 = t$): Aanvaard

Zoals voordien: **hergebruik plaats!**

- ▶ We genereren configuraties c_m één voor één.
- ▶ Wanneer c_m niet meer nodig is, hergebruiken we vrijgekomen plaats.
- ▶ We houden daarenboven ook de diepte van de recursie bij. Dat is, wanneer c_m gegenereerd wordt in de i de recursie:
 - ▶ Voeg (c_1, c_m, i) toe op het eind van de tape en voer $\text{IsBereikbaar}(c_1, c_m, t/2^i)$ uit.
 - ▶ Als deze verwerpt, verwijder (c_1, c_m, i) en vervang c_m door de volgende configuratie.
 - ▶ Als deze aanvaard, bekijk het vorige triple $(*, c_2, i-1)$ op de tape en vervang (c_1, c_m, i) door (c_m, c_2, i) . Voer $\text{IsBereikbaar}(c_m, c_2, t/2^i)$ uit.
 - ▶ Als deze verwerpt, verwijder (c_m, c_2, i) , bekijk het vorige triple $(c_1, *, i-1)$ en bewaar (c_1, c'_m, i) met c'_m devolgende configuratie na c_m .

Heeft IsBereikbaar inderdaad slechts $\mathcal{O}((s(n))^2)$ plaats nodig?

In elke recursieve stap voegen we maximaal één triple (c_1, c_2, t) toe: $\mathcal{O}(s(n))$ plaats.

De diepte van de recursie is $\mathcal{O}(\log(2^{es(n)}))$, aangezien we $2^{es(n)}$ in elke recursie-stap halveren.

Merk op: $\mathcal{O}(\log(2^{es(n)})) = \mathcal{O}(s(n))$.

In totaal: $\mathcal{O}(s(n))\mathcal{O}(s(n)) = \mathcal{O}((s(n))^2)$ plaats!

We veronderstelden dat $s(n)$ in $\mathcal{O}((s(n))^2)$ plaats berekenbaar was.

We kunnen echter de procedure `IsBereikbaar` aanpassen als volgt:

- ▶ Voer een nieuwe parameter s in; en zorg ervoor dat
- ▶ `IsBereikbaar` bekijkt slechts configuraties van lengte s .

Verder vervangen we M door:

1. $s = 1$
2. Voor elke configuratie c van lengte smaller dan $s + 1$:
3. Als `IsBereikbaar`($c_{ini}, c, 2^{d(s+1)}, s + 1$) aanvaard dan
4. aanvaard als $c = c_{accept}$
5. Anders $s = s + 1$ en ga naar stap 2.
6. Verwerp.

Aangezien N in $\mathcal{O}(s(n))$ plaats loopt zullen we zeker verwerpen (voor $s > es(n)$) als we de aanvaardingstoestand nog niet aanvaard hebben. Dit is dus een beslisser en voegt enkel teller toe voor $s(n)$ (logaritmisch in $\mathcal{O}(s(n))$).

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

We hebben reeds opgemerkt dat een TM die $\mathcal{O}(s(n))$ -plaats gebruikt $2^{\mathcal{O}(s(n))}$ mogelijke configuraties heeft.

Dit betekent dat zo'n TM niet meer dan $2^{\mathcal{O}(s(n))}$ stappen kan doen.

- ▶ anders zou een configuratie herhaald worden en dus de TM is geen beslisser.

Beschouw:

$$\text{EXPTIME} = \bigcup_{k \in \mathbb{N}} \text{DTIME}(2^{n^k})$$

Dit impliceert dat:

Stelling

$$\text{PSPACE} \subseteq \text{EXPTIME}$$

We hebben dus:

$$\text{PTIME} \subseteq \text{NP} \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXPTIME}$$

Het is niet geweten of deze inclusies strikt zijn!

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Net zoals bij NP gaan we nu op zoek naar problemen die de complexiteitsklasse PSPACE representeren.

Definitie

Een taal B is **PSPACE-compleet** als:

1. $B \in \text{PSPACE}$ en;
2. Elke taal $A \in \text{PSPACE}$ is reduceerbaar naar B . (**PSPACE-hard**)

- ▶ We gebruiken reducties die **polynomiaal** zijn en dus efficiënt te berekenen zijn.
- ▶ Reducties worden meestal gekozen zodat deze van **lagere complexiteit** zijn dan de klassen die beschouwd wordt
- ▶ Elk niet triviaal probleem in PSPACE is compleet wanneer bijvoorbeeld PSPACE reducties gebruikt zouden worden. (zie voorbeeld op het einde van tijdscomplexiteit slides.)

TBQF is een PSPACE-compleet probleem (we weten al dat het in PSPACE zit).

Beschouw een taal A die beslist wordt door een TM M die gebruik maakt van dn^k plaats ($\mathcal{O}(n^k)$).

We geven nu een **reductie** van A naar TQBF.

Meer bepaald, voor een woord w construeren we een QBF $\phi_{M,w}$ die waar is als en slechts dan M het woord w aanvaardt.

We gebruiken veranderlijken $x_{i,s}$, met i de tape positie en s een element van $Q \cup \Gamma$.

Een configuratie c kan worden beschreven door $\mathcal{O}(n^k)$ dergelijke veranderlijken $x_{i,s}$.

In wat volgt zijn C_1 en C_2 **verzamelingen van veranderlijken** die configuraties beschrijven.

Beschouw twee zo'n verzamelingen: C_1 en C_2 .

Beschouw de parameter $t > 0$.

We gaan een formule $\phi_{C_1, C_2, t}$ definiëren die:

- ▶ die waar is als en slechts dan M in **maximaal t stappen** van (configuratie) C_1 naar (configuratie) C_2 kan gaan
- ▶ en waarvoor C_1 en C_2 **echte** configuraties zijn van de run van M op w .

De uiteindelijke formule ϕ is dan

$$\phi_{M, w} = \phi_{C_{ini}, C_{accept}, h},$$

met $h = 2^{es(n)}$ met e een constante zodat h een bovengrens is op het aantal stappen dat M doet op input w .

We onderscheiden twee gevallen: $t = 1$ (basis geval) and $t > 1$ (induktie)

We noteren de veranderlijken in C_1 met $x_{i,s}$, die in C_2 met $y_{i,s}$.

Voor $t = 1$, $\phi_{C_1, C_2, t}$ moet uitdrukken dat:

- ▶ ofwel $C_1 = C_2$. Dit kan bijvoorbeeld door

$$\bigwedge_{x_{i,s} \in C_1, y_{i,s} \in C_2} (x_{i,s} = y_{i,s}).$$

- ▶ ofwel C_2 kan bereikt worden vanuit C_1 door middel van de **transitierelatie** van M (zoals in de Stelling van Cook-Levin):

$$\bigwedge_{1 < i < dn^k, 1 < j < dn^k} \bigvee_{\substack{a_1, \dots, a_6 \\ \text{is een legal window}}} (x_{i-1, a_1} \wedge x_{i, a_2} \wedge x_{i+1, a_3} \wedge y_{i-1, a_4} \wedge y_{i, a_5} \wedge y_{i+1, a_6}).$$

Notatie:

$$\exists C = \exists x_1, \dots, \exists x_\ell,$$

met $\ell = \mathcal{O}(n^k)$. Dit drukt het bestaan van een configuratie “ C ” uit.

Voor $t > 1$ willen we hetvolgende:

$$\phi_{C_1, C_2, t} = \exists C [\phi_{C_1, C, \lceil \frac{t}{2} \rceil} \wedge \phi_{C, C_2, \lceil \frac{t}{2} \rceil}]$$

Dit verdubbelt echter de grootte van de formule en de uiteindelijke formule ϕ gaat een lengte hebben van $\mathcal{O}(2^{s(n)})$.

Om dit te vermijden gebruiken we een **universele kwantor**.

Notatie: $\forall x \in \{y, z\}[\dots] = \forall x[(x = y \vee x = z) \rightarrow \dots]$

Dan, voor $t > 1$ gebruiken we

$$\phi_{C_1, C_2, t} = \exists C \forall (C_3, C_4) \in \{(C_1, C), (C, C_2)\} [\phi_{C_3, C_4, \lceil \frac{t}{2} \rceil}]$$

Of dus

$$\phi_{C_1, C_2, t} = \underbrace{\exists z_1, \dots, z_\ell}_C \left(\forall \underbrace{u_1, \dots, u_\ell}_{C_3}, \underbrace{v_1, \dots, v_\ell}_{C_4} \right. \\ \left. \left(((C_3 = C_1 \wedge C_4 = C) \vee (C_3 = C \wedge C_4 = C_2)) \rightarrow \phi_{C_3, C_4, \lceil \frac{t}{2} \rceil} \right) \right)$$

In elke stap groeit de formule (additief) met $O(s(n))$ (de grootte van een configuratie).

en we hebben een recursie-diepte van $\log(2^{\mathcal{O}(s(n))}) = \mathcal{O}(s(n))$.

In totaal is de lengte van de formule dus $\mathcal{O}(s^2(n))$ en we kunnen deze in poly-

- ▶ TBQF heeft een “natuurlijke” interpretatie als een **spel tussen twee spelers**.
- ▶ Dit heeft geleid tot de **complexiteitskarakterisatie** van (veralgemeende) spelletjes zoals **schaak, go, ...**
- ▶ Dit is omdat dergelijke spellen een **polynomiale reductie** hebben naar TBQF.

Beschouw de formule:

$$\phi = \exists x_1 \forall x_2 \exists x_3 \dots Q x_k \varphi$$

- ▶ De existentiële en universele kwantoren **wisselen af** (daar kan steeds voor gezorgd worden).
- ▶ We zien ϕ als een **spel tussen twee spelers**:
 - ▶ Speler **E**, die waarden selecteert voor $\exists$ veranderlijken; en
 - ▶ Speler **A**, die waarden kiest voor de $\forall$ veranderlijken.
- ▶ Eén **enkel spel** levert dus een **truth assignment** op voor de veranderlijken.
- ▶ Speler **E** **wint** als φ waar is, speler **A** **wint** als φ niet waar is, en dit voor de bekomen evaluatie.

Voorbeeld

Beschouw

$$\phi = \exists x_1 \forall x_2 \exists x_3 [(x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3})].$$

Een mogelijk spel: E kiest $x_1 = 1$, A $x_2 = 0$, E $x_3 = 1$; E wint.

Merk op: als E $x_1 = 1$ kiest en als A om het even welke waarde kiest voor x_2 , dan als E daaropvolgend $x_3 = \neg x_2$ kiest, dan wint E altijd.

In dit geval zeggen we dat E een **winnende strategie** heeft.

Ongeacht wat speler A doet kan E winnen.

Voorbeeld

Beschouw

$$\phi = \exists x_1 \forall x_2 \exists x_3 [(x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (x_2 \vee \overline{x_3})]$$

Hier heeft **A** een winnende strategie.

Ongeacht wat speler E doet kan A steeds $x_2 = 0$ kiezen, waardoor ϕ steeds onwaar is.

Definitie

FORMULA-GAME = $\{\langle \phi \rangle \mid \text{Speler E heeft een winnende strategie}$
in het spel geassocieerd aan $\phi\}$,

met ϕ zoals voorheen.

Stelling

FORMULA-GAME is PSPACE-compleet.

Stelling

FORMULA-GAME is PSPACE-compleet.

Bewijs: Dit is hetzelfde probleem als TQBF (en dus de reductie is bijna gewoon de identiteit).¹

Een winnende strategie voor E betekent dat φ waar is wanneer de existentiële veranderlijken gekozen worden zoals in de strategie, en dat dus de universele veranderlijken alle mogelijk waarden kunnen hebben (elk mogelijke keuze van A). Dus, φ is waar.

Omgekeerd, wanneer ϕ waar is kunnen we opnieuw hier een winnende strategie voor E uithalen. $\square$

¹Een detail: wanneer TQBF geen alternerende kwantoren heeft kunnen we “dummy” kwantoren toevoegen. Bijvoorbeeld: $\exists x_1 \exists x_2 \phi$ wordt $\exists x_1 \forall y_1 \exists x_2 (\phi \wedge (y_1 \vee \bar{y}_1))$.

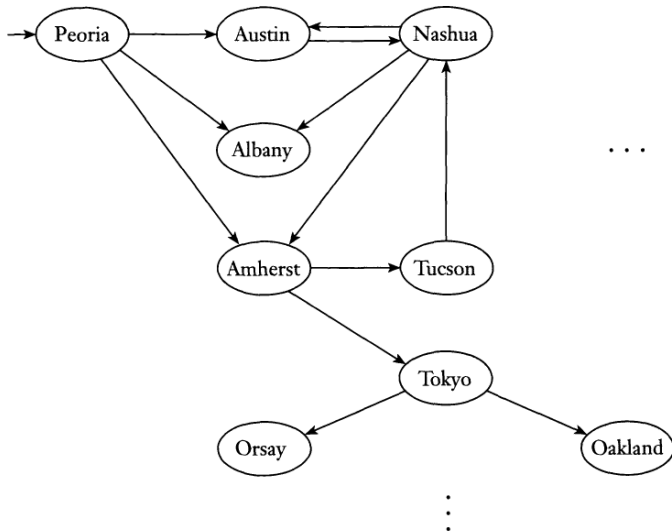
Spel van **geografie**:

- ▶ Speler I noemt een stad c
- ▶ Speler II noemt een stad d die begint met de laatste letter van de stad c
- ▶ Speler I doet hetzelfde, ditmaal voor d
- ▶ Enzovoorts.

Geen stadsnamen mogen meerdere keren gekozen worden

Als een speler geen zet meer kan doen dan verliest hij/zij het spel.

Dit spel heeft dus te maken met het doorlopen van een graaf:

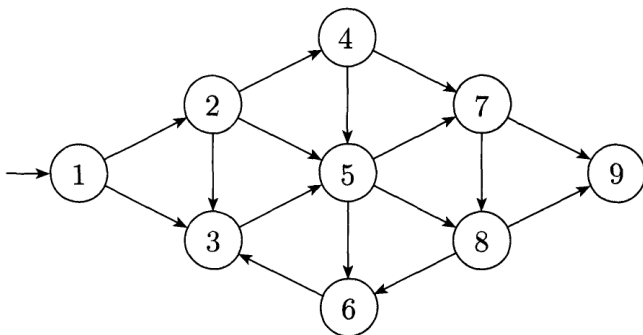


Een **abstractie** van dit spel is als volgt:

- ▶ Beschouw een **gerichte graaf** met een **beginknoop**.
- ▶ Speler I beweegt vanuit deze knoop.
- ▶ Vervolgens beweegt Speler II van de knoop gekozen door Speler I, enz.

Opnieuw, geen knopen mogen herhaald worden (**enkelvoudig pad**).

Het doel is om een speler in zo'n positie te krijgen dat geen verdere stappen gezet kunnen worden.



Speler I heeft een winnende strategie (kies 3, dan 6)

Definitie

$GG = \{ \langle G, b \rangle \mid \text{Speler I heeft een winnende strategie voor} \\ \text{het veralgemeende geografie spel op de graaf } G \\ \text{beginnend vanuit knoop } b \}$

Stelling

GG is PSPACE-compleet.

We beschouwen de volgende TM M .

Op input $\langle G, b \rangle$ met G een gerichte graaf een knoop b :

1. Als b geen uitgaande knopen heeft, dan verliest speler I. Verwerp.
2. Stel, $b_1, \dots, b_k$ zijn knopen zodat (b, b_i) bogen zijn. Verwijder knoop b en alle in en uitgaande bogen. Noem de nieuwe graaf G' .
3. Voor elke b_i roep M **recursief** aan op $\langle G', b_i \rangle$.
4. Als deze alle aanvaarden, dan heeft Speler II een winnende strategie. Verwerp.
5. Anders, Speler I heeft een winnende strategie en aanvaard.

Recursie diepte m (aantal knopen), elke keer een knoop erbij. Dus M heeft $\mathcal{O}(m)$ plaats nodig.

We maken een reductie van FORMULA-GAME.

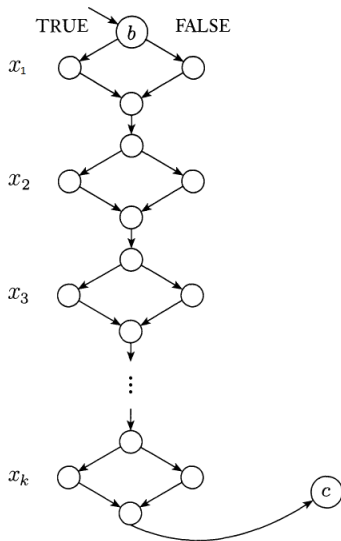
Beschouw $\phi = \exists x_1 \forall x_2 \exists x_3 \dots \exists x_k \varphi$.

We veronderstellen dat ϕ *begint en eindigt* met $\exists$ en bovendien $\exists, \forall$ wisselen elkaar af.

We veronderstellen ook dat φ in CNF is.

We construeren een graaf G zodat Speler I een winnende strategie heeft voor GG op G als en slechts dan ϕ waar is.

GG is PSPACE-hard: truth assignment



- ▶ Eén diamant per veranderlijke.
- ▶ Speler I start (links betekent $x_1 = 1$, rechts $x_1 = 0$).
- ▶ Speler Player II moet naar beneden, dan is het opnieuw aan Speler I.
- ▶ Vervolgens moet Speler II naar de diamant voor x_2 en een keuze maken, enz.
- ▶ Op het eind hebben we $\exists x_k$, Speler I maakt een keuze, Speler II gaat naar beneden.
- ▶ Dus Speler I moet de laatste stap doen, naar knoop c .

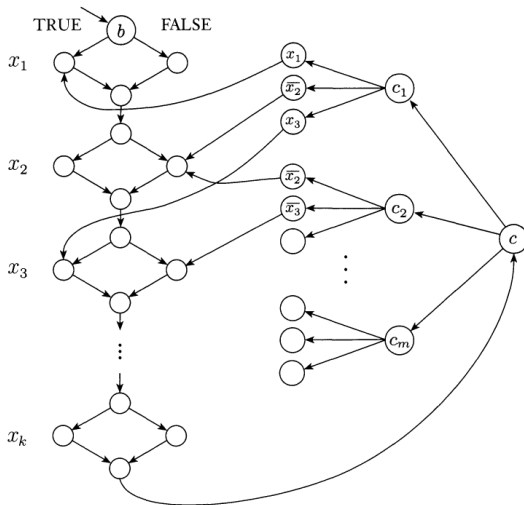
Eén zo een spel levert een truth assignment ν op van de veranderlijken $x_1, \dots, x_k$

We bouwen nu verder aan G om zo φ erbij te betrekken

We willen hetvolgende:

- ▶ Speler II is aan zet. Deze moet een knoop kiezen die een clause representeert.
 - ▶ Bogen van c naar clause knopen.
- ▶ Speler I is aan zet. Deze moet een knoop kiezen die een literal representeert.
 - ▶ Bogen van de clause knopen naar de literal knopen.
 - ▶ We hebben literal knopen, allen apart voor elke clause.
- ▶ Speler II is aan zet. We relateren literals met de gekozen truth assignment.
 - ▶ Een boog van positieve literal x_i naar linkse knoop in diamant voor x_i .
 - ▶ Een boog van negatieve literal $\bar{x}_i$ naar rechtse knoop in diamant voor x_i .

$$\phi = \exists x_1 \forall x_k \dots \exists x_k [(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3} \vee \dots) \wedge \dots]$$



We tonen aan dat als ϕ waar is dat dan Speler I een winnende strategie heeft in G, b .

Als ϕ waar is:

- ▶ Speler I kiest $\nu(x_1) \in \{0, 1\}$ zo dat $\forall x_2 \exists x_3, \dots, \exists x_k \varphi(\nu(x_1), x_2, \dots, x_k)$ waar is.
- ▶ Speler II kiest $\nu(x_2) \in \{0, 1\}$. Er bestaat $\nu(x_3) \in \{0, 1\}$ zodat $\forall x_4 \exists x_5, \dots, \exists x_k \varphi(\nu(x_1), \nu(x_2), \nu(x_3), x_4, \dots, x_k)$ waar is. Speler I kiest deze waarde.
- ▶ Enzovoorts. We eindigen met een valuatie $\nu : X \rightarrow \{0, 1\}$ zodat $\varphi(\nu(x_1), \nu(x_2), \nu(x_3), \dots, \nu(x_k))$ waar is.
- ▶ Aangezien φ een conjunctie van clauses $C_1, \dots, C_r$ is moet elke clause waar zijn onder ν . Dus, welke clause knoop Speler II ook kiest, er is altijd een literal die waar is in de gekozen clause.
- ▶ Speler I kiest dan die literal.
- ▶ Speler II moet nu ofwel een boog van x_i naar linkse knoop in diamant, ofwel $\bar{x}_i$ naar rechtse knoop in diamant kiezen. Echter, $\nu(x_i) = 1$ ofwel $\nu(\bar{x}_i) = 1$ en dus Speler I heeft die knopen (links of rechts) al geselecteerd.
- ▶ Speler II kan dus geen zet meer doen.

We tonen aan dat als ϕ onwaar dat Speler I geen winnende strategie heeft in G, b .

Stel dat ϕ onwaar is:

- ▶ Wat Speler I ook kiest, Speler II kan ervoor zorgen dat onder de gekozen truth assignment φ niet waar is.
- ▶ Speler II kiest een clause knoop die niet waar is.
- ▶ Welke literal knoop Speler I ook kiest, deze gaat steeds niet waar zijn onder de gekozen truth assignment.
- ▶ Stel Speler I kiest een x_i , dan volgt Speler II de boog van x_i naar de linkse knoop in het diamant. Aangezien $x_i = 0$ is deze knoop nog niet bezocht. Echter, Speler I zit nu vast.
- ▶ Stel Speler I kiest een $\bar{x}_i$, dan volgt Speler II de boog van $\bar{x}_i$ naar de rechtse knoop in het diamant. Aangezien $\bar{x}_i = 0$ is deze knoop nog niet bezocht. Echter, Speler I zit nu vast.

De reductie is duidelijk berekenbaar in PTIME.



- ▶ DSPACE en NSPACE en voorbeelden (Sipser, begin hoofdstuk 8)
- ▶ De stelling van Savitch (Sipser 8.1)
- ▶ PSPACE en NPSPACE (Sipser 8.2)
- ▶ PSPACE-compleetheid en voorbeelden van dergelijke problemen, juistheid van reducties (Sipser 8.3)

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Tot nu toe hebben we tijd- en plaatscomplexiteit bekeken waarbij we **ten minste lineaire tijd** (nodig om de input te lezen) of **linear plaats** (nodig om de input op te slaan) veronderstellen.

- ▶ Sublineaire tijd is interessant maar kan alleen leiden tot approximatie algoritmen (input kan niet volledig gelezen worden).
- ▶ Sublineaire plaats is interessant (input kan gelezen worden maar niet alles ervan kan worden opgeslagen).

Om sublineaire plaatscomplexiteit te bestuderen gebruikt men een **variatie van TMs**.

TM met twee tapes: één **read-only** input tape en één **werk-tape**.

Definitie

Een (deterministische) **2-tape Turing machine met read-only input en werk-tape** is gedefinieerd zoals een gewone 2-tape TM maar waar voor **elke** $\delta(q, a_1, a_2) = (q', b_1, b_2, d_1, d_2)$ met $q' \in Q$, $b_1, b_2 \in \Gamma$ and $d_1, d_2 \in \{\leftarrow, \square, \rightarrow\}$ we hebben dat:

1. $a_1 = b_1$ (de eerste tape is read only);
2. Als $a_1 = \square$ dan $d_1 = \leftarrow$ zodat de TM op het input woord blijft;
3. Op beide tapes kunnen we niet verder naar rechts dan het $\vdash$ symbool (zoals voordien).

Zoals voorheen:

Definitie

- ▶ Voor een 2-tape **deterministische** TM M met read-only input en werk-tape is de **plaatscomplexiteitsfunctie** $s_M(n)$ het maximaal aantal cellen dat M scant op de werktape op elk mogelijke input woord w van lengte n .
- ▶ Voor een 2-tape **niet-deterministische** TM M met read-only input en werk-tape is dit het maximaal aantal cellen dat M scant op de werktape, en dit **over alle takken** van de berekeningsbomen van M op elk mogelijke input woord w van lengte n .
- ▶ Zoals steeds: M gebruikt $\mathcal{O}(s(n))$ plaats als $s_M(n)$ van orde $\mathcal{O}(s(n))$ is.

Wanneer $s(n) \geq n$ maakt het niet uit of we een single tape TM of TM met read-only input en werk-tape beschouwen (voor plaatscomplexiteit)! (Herinner simulatie.)

We definiëren:

Definitie

- ▶ **LOGSPACE** de klasse van talen die beslisbaar zijn door een deterministische TM met een read-only input en werk-tape die logaritmisch veel plaats gebruikt:

$$\text{LOGSPACE} = \text{DSPACE}(\log n).$$

- ▶ **NLOGSPACE** de klasse van talen die beslisbaar zijn door een niet-deterministische TM met een read-only input en werk-tape die logaritmisch veel plaats gebruikt:

$$\text{NLOGSPACE} = \text{NSPACE}(\log n).$$

Deze definities hebben alleen maar zin voor TMs met een read-only input en werk-tape!

Voorbeeld

Beschouw de taal $L = \{0^k 1^k \mid k \geq 0\}$.

De TMs die we eerder gezien hebben voor palindromen kunnen niet direct gebruikt worden:

- ▶ Ofwel moeten ze input kunnen veranderen (1-tape TM).
- ▶ Ofwel moet de input gekopieerd worden (2-tape TM).

Echter, we kunnen een TM gebruiken die op de werk tape:

- ▶ Eerst **aantal nullen telt** (en verwerpt als er na een 1 een 0 komt). Op de werk tape houden we dus een **teller** bij van maximaal $\log(n)$ bits.
- ▶ Vervolgens tellen we het **aantal enen**. Op de werk tape houden we dus een **teller** bij van maximaal $\log(n)$ bits.
- ▶ We testen dan of deze tellers gelijk zijn. Zoja, aanvaard, anders verwerp.

Dit is duidelijk in LOGSPACE.

Voorbeeld

Beschouw de taal $L = \{0^k 1^k \mid k \geq 0\}$.

De TMs die we eerder gezien hebben voor palindromen kunnen niet direkt gebruikt worden:

- ▶ Ofwel moeten ze input kunnen veranderen (1-tape TM).
- ▶ Ofwel moet de input gekopieerd worden (2-tape TM).

Echter, we kunnen een TM gebruiken die op de werk tape:

- ▶ Eerst **aantal nullen telt** (en verwerpt als er na een 1 een 0 komt). Op de werk tape houden we dus een **teller** bij van maximaal $\log(n)$ bits.
- ▶ Vervolgens tellen we het **aantal enen**. Op de werk tape houden we dus een **teller** bij van maximaal $\log(n)$ bits.
- ▶ We testen dan of deze tellers gelijk zijn. Zoja, aanvaard, anders verwerp.

Dit is duidelijk in LOGSPACE.

Herinner:

Definitie

$$\text{PATH} = \{ \langle G, s, t \rangle \mid G \text{ is een gerichte graaf met een pad van } s \text{ naar } t \}$$

We hebben al gezien dat $\text{PATH} \in \text{PTIME}$ zit.

Echter, dat algoritme gebruikte **linear plaats**.

We tonen aan dat PATH in NLOGSPACE zit

Het is niet geweten of PATH ook in LOGSPACE zit!

Een NLOGSPACE algoritme voor PATH:

$M = \text{Op input } \langle G, s, t \rangle \text{ met } G \text{ een gerichte graaf met beginknoop } s \text{ en eindknoop } t:$

1. Tel het **aantal** knopen n in G en schrijf dit aantal (binair) op de werk tape.
2. Schrijf ook een **pointer** p op de werk tape. Initieel p verwijst naar de positie van de knoop s op de input tape. In het algemeen stelt v_p de knoop voor naarwaar p verwijst.
3. Initialiseer een **teller** op 0, ook op de werk tape.
4. **Herhaal** todat $v_p = t$, ofwel de teller gelijk is n :
5. Kies (**niet-determinisch**) een nieuwe pointer q die verwijst naar een knoop v_q van de input tape.
6. Als (v_p, v_q) geen boog is, **verwerp** dan deze keuze.
7. Als (v_p, v_q) wel een boog is, **vervang** p met q .
8. **Tel** in ieder geval 1 **op** bij de teller.
9. Als de pointer p naar t verwijst, **aanvaard**.

In elke tak in de berekeningsboom houden we slechts 2 pointers and 2 tellers by, elk van grootte $\log(n)$. Als er een pad bestaat, bestaat er één van lengte maximaal n dus een tak zal dit pad kiezen. Als er geen pad bestaat, dan zal elke keuze van n knopen niet van s naar t gaan en dus zullen alle taken verwerpen.

Een NLOGSPACE algoritme voor PATH:

$M =$ Op input $\langle G, s, t \rangle$ met G een gerichte graaf met beginknoop s en eindknoop t :

1. Tel het **aantal** knopen n in G en schrijf dit aantal (binair) op de werk tape.
2. Schrijf ook een **pointer** p op de werk tape. Initieel p verwijst naar de positie van de knoop s op de input tape. In het algemeen stelt v_p de knoop voor naarwaar p verwijst.
3. Initialiseer een **teller** op 0, ook op de werk tape.
4. **Herhaal** todat $v_p = t$, ofwel de teller gelijk is n :
5. Kies (**niet-determinisch**) een nieuwe pointer q die verwijst naar een knoop v_q van de input tape.
6. Als (v_p, v_q) geen boog is, **verwerp** dan deze keuze.
7. Als (v_p, v_q) wel een boog is, **vervang** p met q .
8. **Tel** in ieder geval 1 **op** bij de teller.
9. Als de pointer p naar t verwijst, **aanvaard**.

In elke tak in de berekeningsboom houden we slechts 2 pointers and 2 tellers by, elk van grootte $\log(n)$. Als er een pad bestaat, bestaat er één van lengte maximaal n dus een tak zal dit pad kiezen. Als er geen pad bestaat, dan zal elke keuze van n knopen niet van s naar t gaan en dus zullen alle taken verwerpen.

Algemene opmerkingen:

- ▶ In logaritmische plaats kunnen we een **constant aantal pointers** bijhouden die verwijzen naar posities op de input tape
- ▶ In logaritmische plaats kunnen we een **constant aantal tellers** bijhouden van grootte maximaal n (binair gerepresenteerd)
- ▶ In LOGSPACE kunnen we dus problemen oplossen met een **constant aantal pointers en tellers**.
- ▶ in NLOGSPACE kunnen we problemen oplossen die een **constant aantal pointers en tellers** nodig hebben om een oplossing van het probleem te verifiëren.

We kunnen, net zoals bij NP, een andere definitie geven voor NLOGSPACE, gebaseerd op **verifiërs**:

Definitie

- ▶ Een LOGSPACE **verifiër** voor een taal L is een deterministische TM V met
 - ▶ een read-only input tape;
 - ▶ een werk-tape;
 - ▶ een **read-once** tape,

zodat

$$L = \{w \mid V \text{ aanvaardt } \langle w, c \rangle \text{ voor een woord } c \text{ op de read-once tape}\}.$$

met certificate c van polynomiale lengte ($p(|w|)$) en zodat V slechts logaritmisch veel plaats gebruikt.

In de oefeningen wordt de equivalentie van de twee definities voor NLOGSPACE bekeken.

De Stelling van Savitch is ook waar voor TM met een read-only input en werktape.

Een kleine aanpassing is nodig:

Definitie

In een configuratie bewaren we de toestand, de posities van de twee koppen en de inhoud van de werktape.

Veronderstel $s(n) \geq \log(n)$ voor NTM M .

Dit levert $|Q|ns(n)|\Gamma|^{ds(n)} = n2^{\mathcal{O}(s(n))}$ configuraties op.

Een configuratie heeft slechts $\log(n2^{\mathcal{O}(s(n))}) = \log(n) + \mathcal{O}(s(n)) = \mathcal{O}(s(n))$ plaats nodig. Dus,

Stelling (Savitch)

Als $s(n) \geq \log(n)$, dan

$$NSPACE(s(n)) \subseteq DSPACE(s^2(n)).$$

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Definitie

Een taal B is **NLOGSPACE-compleet** als:

1. $B \in \text{NLOGSPACE}$ en;
2. elke taal $A \in \text{NLOGSPACE}$ kan gereduceerd worden naar B (**NLOGSPACE-hard**).

Maar welke reducties te gebruiken?

Dit moet **zwakker** zijn dan reducties die berekenbaar zijn in NLOGSPACE!

Aangezien het niet geweten is of $\text{LOGSPACE} = \text{NLOGSPACE}$ worden LOGSPACE-reducties gebruikt.

Hoe worden zo'n zwakke reducties gedefinieerd?

Definitie

Een functie $f : \Sigma^* \rightarrow \Sigma^*$ is **LOGSPACE-berekenbaar** als er een 3-tape Turing machine bestaat met input tape (read-only), werk-tape, **write-only output tape**, zodat wanneer w op de input tape staat, de machine stopt met $f(w)$ op de output tape en de **gebruikte plaats op de werk-tape** is $\mathcal{O}(\log(n))$, voor $n = |w|$.

(Op een write-only tape kan niet naar links gegaan worden (dus niet lezen wat er al geschreven is)).

Definitie

Een taal A is **LOGSPACE-reduceerbaar** tot een taal B , genoteerd als $A \leq_L B$, indien er een LOGSPACE-berekenbare functie $f : \Sigma^* \rightarrow \Sigma^*$ bestaat zodat

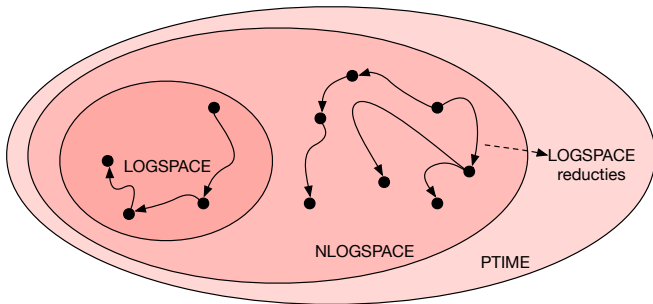
$$w \in A \iff f(w) \in B.$$

Definitie

Een taal B is NLOGSPACE-compleet als:

1. $B \in \text{NLOGSPACE}$ en;
2. elke taal $A \in \text{NLOGSPACE}$ is LOGSPACE-reduceerbaar tot de taal B .

We kijken even na of dergelijke reducties ook de gewenste eigenschappen hebben.



LOGSPACE is **gesloten** onder LOGSPACE-reducties

Propositie

Als $A \leq_L B$ en $B \in \text{LOGSPACE}$ dan $A \in \text{LOGSPACE}$.

(Zie volgende slides.)

Een gevolg hiervan is:

Propositie

Als een NLOGSPACE-hard probleem in LOGSPACE zit, dan is

$$\text{LOGSPACE} = \text{NLOGSPACE}.$$

Inderdaad, neem een taal A in NLOGSPACE en een NLOGSPACE-hard probleem B . Dan $A \leq_L B$ en dus als $B \in \text{LOGSPACE}$ dan zit, volgens de vorige propositie, ook A in LOGSPACE. (We hadden hetzelfde voordien met $\leq_P$, PTIME en NP).

We hebben ook **transitiviteit**.

Propositie

Als $A \leq_L B$ en $B \leq_L C$ dan $A \leq_L C$.

We kunnen echter niet gewoon de reducties samenstellen!

Hoe groot kan de output zijn van een LOGSPACE-berekenbare functie f ?

- ▶ Zij M_f de 3-tape TM die f berekent.
- ▶ M_f is een beslisser en dus geen enkele configuratie van M_f op input w kan herhaald worden.
- ▶ We hebben eerder gezien dat er $n2^{\mathcal{O}(\log(n))} = \mathcal{O}(n^c)$ configuraties zijn voor een constante c .
- ▶ M_f doet dus maximaal $\mathcal{O}(n^c)$ stappen.
- ▶ M_f kan slechts een woord van lengte $\mathcal{O}(n^c)$ op de output tape schrijven.

Dus simpelweg samenstellen werkt niet aangezien TM M_{gof} :

- ▶ Het input woord w op de input tape heeft,
- ▶ dan $f(w)$ op de werk-tape moet schrijven (**mogelijke poly space!**)
- ▶ dan $g(f(w))$ moet berekenen en op de output tape schrijven.

Beter idee: bereken $f(w)$ “on-the-fly”.

Propositie

Als $A \leq_L B$ en $B \leq_L C$ dan $A \leq_L C$.

Bewijs: Stel dat M_f de TM is die de LOGSPACE-reductie van A naar B berekent, en dat M_g de TM is die de LOGSPACE-reductie van B naar C berekent.

We beschouwen de volgende TM: Input w

1. Zet een **teller** t (binair) op het begin van werk tape, initieel is deze 1.
2. Voer een **aangepaste versie** van M_f uit op w :
 - ▶ Voer M_f uit op w maar in plaats van $f(w)$ op de output tape te zetten, schrijf alleen de **eerste cell** $f(w)_1$ **van de output van M_f** op w na de teller t op de werk-tape.
 - ▶ Tesamen met wat M_f **nodig heeft**, hebben we dus t en **één symbool** $f(w)_1$ op de werk-tape.
 - ▶ Als de teller dus logaritmisch in grootte is, bevat de werk-tape **logaritmisch veel cellen**.
 - ▶ **Verwijder** alles behalve t and $f(w)_1$ van de werk-tape.

3. Voer vervolgens M_g **uit op** $f(w)_1$ (hier staat de kop van M_g op).
 - ▶ Als dit output genereert, **schrijf** deze op de output tape (dit gaat $g(f(w))_1$ zijn).
 - ▶ Kijk dan hoe M_g over de input loopt (links, rechts, blijven staan). **Update** t aan de hand hiervan (decrement, geen verandering, increment).
 - ▶ Op de tape staat nu t (updated), $f(w)_1$, plus wat M_g nodig had om een stap te doen. Dus, **logaritmisch veel cellen**.
 - ▶ **Verwijder** $f(w)_1$ van de werk-tape,
4. Herhaal stap 2, door M_f **opnieuw uit te voeren**, maar nu voor het t -de-symbool.
 - ▶ Ttz, de werk-tape bevat t , $f(w)_t$, en wat M_g nodig had om de vorige stap te doen en wat M_f nodig heeft om $f(w)_t$ te berekenen.
 - ▶ Nog steeds **logaritmisch** als de teller van logaritmische lengte is.
 - ▶ **Verwijder** alles wat M_f nodig had.

5. Herhaal stap 3, door M_g uit te voeren (de kop van M_g staat op t -symbool van “input” $f(w)$.
 - ▶ Op de werktape staat nu, t (updated), $f(w)_t$, en nieuwe werk-tape inhoud voor stap van M_g . Nog steeds **logaritmisch** als teller logaritmisch groot is.
 - ▶ Als M_g output maakt, is dit hetvolgende symbool van $g(f(w))$. **Schrijf** dit op output-tape.
 - ▶ **Verwijder** $f(w)_t$.
6. **Blijf dit doen** zolang M_g elementen van $f(w)$ vraagt. Op het eind zal dus $g(f(w))$ op de output-tape staan.

We merken nu op dat de teller t van grootte $\mathcal{O}(\log(f(w))) = \mathcal{O}(\log(n^c)) = \mathcal{O}(\log(n))$ is.

Op geen enkel moment bevat de tape meer dan een logaritmisch aantal symbolen!

Propositie

Als $A \leq_L B$ en $B \in \text{LOGSPACE}$ dan $A \in \text{LOGSPACE}$.

Bewijs. Het transitiviteitsbewijs laat zien dat twee LOGSPACE-berekenbare functies kunnen worden samengesteld zodat deze samenstelling opnieuw een LOGSPACE-berekenbare functies is.

Beschouw de functie $g(w) = 1$ als $w \in B$ and $g(w) = 0$ als $w \notin B$. Aangezien $B \in \text{LOGSPACE}$ zit is g een LOGSPACE-berekenbare functie.

Beschouw nu de LOGSPACE-berekenbare functie die $g \circ f$ berekent, met f the LOGSPACE-reductie van A naar B .

Dan, $g(f(w)) = 1$ als en slechts dan $f(w) \in B$ als en slechts dan $w \in A$.

Dus, $A \in \text{LOGSPACE}$.



Stelling

PATH is NLOGSPACE-complete.

Bewijs: We hebben al gezien dat PAD in NLOGSPACE zit

Neem nu een taal A in NLOGSPACE.

Neem M , een NLOGSPACE machine die A beslist.

Het woord w is de input.

We construeren een graaf $\langle G, s, t \rangle$ dmv een LOGSPACE-berekenbare functie zodat M het woord w aanvaardt als en slechts als er een pad is van s naar t in G .

Gegeven de TM M voor A in LOGSPACE en input w van grootte $n = |w|$.

De reductie gaat eerst **alle configuraties** van M op w genereren:

- ▶ Een knoop in $\langle G, s, t \rangle$ is zo een configuratie.
- ▶ Herinner, configuraties bevatten een toestand, posities van twee koppen, en de inhoud van de werk-tape.
- ▶ De lengte van een configuratie is $c \cdot \log(n)$, voor een zeker constante c (zie slide 11).

We genereren die configuraties als volgt:

- ▶ Simpelweg **genereer alle woorden** van lengte $c \log(n)$ op de werk-tape, één per één.
- ▶ Plaats die op de **output tape** als dit een **mogelijke configuratie** is van M op w (syntactische test).

We hebben hier duidelijk slechts LOGSPACE cellen voor nodig op de werk-tape.

Op een zelfde manier **genereren we bogen** die overeen komen met **transities tussen configuraties**:

- ▶ **Genereer nu alle koppels van configuraties** (c_1, c_2)
- ▶ **Schrijf** die als een boog op de **output tape** als M kan gaan van c_1 naar c_2 in één stap (opnieuw simpele test dmv transitiefunctie).

Op de output-tape staan dus al de knopen en bogen van de configuratiegraaf van M op w .

Rest ons nog op de **initiële en aanvaardingsconfiguratie als s en t** op de output tape te schrijven.

- ▶ Maw, we schrijven $\vdash q_{ini}w$ op de output-tape (w staat op input-tape) als s .
- ▶ We veronderstellen dat M één aanvaardingsconfiguratie heeft van de vorm

$$\vdash q_{accept} \sqcup \dots \sqcup$$

van lengte $c \log(n)$. Schrijf deze op de output-tape als t . □

Dit is een LOGSPACE-reductie van A naar PATH.

Herinner 2SAT? Zoals 3SAT NP-compleet is, is 2SAT NLOGSPACE-compleet.

Dit toont opnieuw het belang aan van propositionele formules en SAT solvers, 2SAT solvers enz.

Stelling

2SAT is NLOGSPACE-compleet.

Bewijs. We gebruiken dat $\text{NLOGSPACE} = \text{co-NLOGSPACE}$ (De Stelling van Immerman-Szelepcsényi, zie later).

We kunnen dus evengoed aantonen dat $\overline{2\text{SAT}}$ in NLOGSPACE zit.

Merk op dat ook $\overline{\text{PATH}}$ NLOGSPACE-compleet is en we kunnen dus een LOGSPACE-reductie $\overline{\text{PATH}} \leq_L 2\text{SAT}$ gebruiken om aan te tonen dat 2SAT ook NLOGSPACE-hard is.

$\overline{2\text{SAT}}$ in NLOGSPACE?

We hebben een PTIME algoritme voor 2SAT gezien:

- ▶ Maak de implicatiegraaf.
- ▶ Aanvaard als er **geen enkele** literal x bestaat met een pad van x naar $\bar{x}$, en omgekeerd.

Dus, voor $\overline{2SAT}$ moeten we testen of er **een** literal x **bestaat** met een pad van x naar $\bar{x}$ en omgekeerd.

We gaan het algoritme voor PATH aanpassen...

$M = \text{Op input } \langle \varphi \rangle \text{ met } \varphi \text{ een 2SAT formule:}$

1. Tel het **aantal** literals n in ϕ en schrijf $2n$ (binair) op de werk tape.
2. Kies **niet-deterministisch** een literal x in ϕ . Doe dit door middel van een pointer p op de werk-tape die de positie van de literal in $\langle \varphi \rangle$ bepaalt. Duid met x_p de literal aan waarnaar p verwijst. Scan de input en laat $\bar{p}$ de pointer zijn die naar $\bar{x}_p$ verwijst.
3. Initialiseer een **teller** op 0, ook op de werk-tape.
4. **Herhaal** todat $x_p = \bar{x}$, ofwel de teller gelijk is $2n$:
5. Kies (**niet-deterministisch**) een nieuwe pointer q die verwijst naar een literal x_q van de input tape.
6. Als (x_p, x_q) geen boog is in de implicatiegraaf, **verwerp** dan deze keuze.
7. Als (x_p, x_q) wel een boog is, **vervang** p met q .
8. **Tel** in ieder geval 1 **op** bij de teller.
9. Als de pointer p niet naar $\bar{x}$ verwijst, **verwerp**.
10. Herhaal stappen 4.-8. voor $\bar{p}$ and $x_{\bar{p}} = x$.
11. Als de pointer $\bar{p}$ naar x verwijst, aanvaard.

Dus, $\overline{2SAT}$ is in NLOGSPACE en dus 2SAT is ook in NLOGSPACE (NLOGSPACE=co-NLOGSPACE).

We maken een LOGSPACE-reductie van $\overline{\text{PATH}}$ naar 2SAT.

We moeten een LOGSPACE-berekenbare functie $f : \Sigma^* \rightarrow \Sigma^*$ maken zodat $w \in \overline{\text{PATH}}$ als en slechts dan $f(w) \in \text{2SAT}$:

- ▶ Neem $w = \langle G, s, t \rangle$ op de input-tape.
- ▶ Voor elke boog (a, b) in G zet clause $(\bar{a} \vee b)$ op de output tape.
- ▶ Zet ook s en $\bar{t}$ als clause op de output tape.

Is dit een reductie?

Stel $w \in \overline{\text{PATH}}$ en dus er is geen pad van s naar t in $G = (V, E)$.

Splits V op in twee sets S en T , met $s \in S$ en $t \in T$ en er is geen boog van een knoop van S naar T .

Definieer $\nu : V \rightarrow \{0, 1\}$ zodat $\nu(a) = 1$ als $a \in S$ en $\nu(b) = 0$ als $b \in T$.

We weten

$$\phi = s \wedge \bar{t} \wedge \bigwedge_{(a,b) \in E} \bar{a} \vee b = (s \wedge \bigwedge_{(a,b) \in E_S} \bar{a} \vee b) \wedge (\bar{t} \wedge \bigwedge_{(a,b) \in E_T} \bar{a} \vee b),$$

met E_S bestaande uit s en ook alle $(a, b) \in E$ met $a \in S$. En E_T bevat al de rest.

Dan, $\nu(s) = 1$ en voor alle $(a, b) \in E_S$, $\nu \models (\bar{a} \vee b)$ omdat $b \notin T$. Ook, $\nu(\bar{t}) = 1$ en voor alle $(a, b) \in E_T$, $\nu \models (\bar{a} \vee b)$ omdat $a \notin S$.

Omgekeerd, stel er een truth assignment ν bestaat zodat $\nu \models \varphi$.

Veronderstel dat er een pad p van s naar t bestaat.

Elk zo'n pad p komt overeen met een reeks bogen (a_i, b_i) startend in s en eindigend in t , en ook met clauses $\bar{a}_i \vee b_i$ en s en $\bar{t}$.

En dus $\nu \models s$, $\nu \models \bar{t}$ en $\nu \models (\bar{a}_i \vee b_i)$.

Zoals we eerder hebben gezien (2SAT algoritme), moet echter $\nu \models t$ ook waar zijn. Een contradictie. $\square$

Plaatscomplexiteit

De Stelling van Savitch: $PSPACE \subseteq NPSPACE$

Tijd versus plaatscomplexiteit

PSPACE-compleetheid

Logaritmische plaatscomplexiteit: LOGSPACE en NLOGSPACE

NLOGSPACE-complete problemen

De Stelling van Immerman-Szelepcsényi: $NLOGSPACE = co-NLOGSPACE$

Stelling (Immerman-Szelepcsényi)

$NLOGSPACE = co-NLOGSPACE$

Bewijs: We tonen aan dat $\overline{PATH}$ (complement probleem) in $NLOGSPACE$ zit.

Omdat $PATH$ $NLOGSPACE$ -compleet is, is dit voldoende.

- ▶ Waarom? Werk dit zelf uit in de oefeningen.

Herinner, de input van $PATH$ is $\langle G, s, t \rangle$.

We moeten nu een NTM M maken (logaritmische ruimte) dewelke $\langle G, s, t \rangle$ aanvaardt als en slechts dan **er geen pad is van s naar t** .

We kunnen niet alle knopen bewaren op de werk-tape die bereikt kunnen worden vanuit s , en dan testen of t in die verzameling zit! (Er zijn een lineair aantal dergelijke knopen.)

In plaats daarvan gebruiken we een **teller**.

Voor een graaf $G = (V, E)$ met $m = |V|$ aantal knopen.

Definieer c als het **aantal knopen die bereikt kunnen worden vanuit s** .

Als we c kennen dan kunnen efficiënt bepalen dat er **geen pad** is van s naar t .

We laten nu zien hoe dat kan. Daarna leggen we uit hoe c berekend kan worden.

$M =$ Op input G, s, t en c doe het volgende:

1. $d := 0$
2. Voor elke knoop u in G :
3. Kies, op een niet-deterministische manier, of u bereikbaar is vanuit s door middel van een pad van lengte $\leq m$.
4. Als de keuze “ja” was, dan volg (opnieuw non-deterministisch) een pad van lengte $\leq m$ vanuit s . Als de keuze “nee” was doe niets, ttz, ga naar de volgende knoop op regel 2.
5. Verwerp als dit pad niet eindigt in u . (Contradictie met keuze “ja”)
6. Verwerp als $u = t$. (Er is een pad van s naar t !)
7. $d := d + 1$ (Tel het aantal knopen waarvan de bereikbaarheid vanuit s gechecked is)
8. Als $d \neq c$ verwerp, anders aanvaard.

Voor aanvaarden: Er is een run die (i) alle c knopen die bereikbaar zijn vanuit s identificeert, én (ii) geen van die knopen t is. Maw, er is geen pad van s naar t .

We hebben alleen een teller nodig (d), aantal knopen m en plaats om een pad, knoop per knoop, te maken.

Laat A_i de verzameling zijn van knopen die bereikbaar zijn vanuit s in **maximaal i stappen**.

Dus, $A_0 = \{s\}$ en $A_i \subseteq A_{i+1}$.

Definieer $c_i = |A_i|$ (en dus $c = c_m$).

We gaan nu c_{i+1} berekenen a.h.v. c_i .

Om te testen of $v \in A_{i+1}$ gebruiken we het algoritme van voordien wanneer c_i gekend is.

Veronderstel dat we c_i kennen; dan berekenen we c_{i+1} als volgt:

1. $c_{i+1} := 1$ (We weten dat $s \in A_{i+1}$ zit)
2. Voor elke knoop $v \neq s \in G$: (We checken of $v \in A_{i+1}$ zit)
3. $d := 0$ (We gaan vertices in A_i reconstrueren)
4. Voor elke knoop $u \in G$: (Zit u in A_i ?)
5. Kies op een niet-deterministische manier of $u \in A_i$.
6. Als de keuze "ja" is, dan volg een pad vanuit s van lengte maximaal i .
Keuze "nee", ga naar volgende knoop.
7. Verwerp als dit pad niet eindigt in u .
8. $d = d + 1$
9. Als (u, v) een boog is $c_{i+1} = c_{i+1} + 1$ en ga naar de volgende knoop v .
(v zit in A_{i+1})
10. Als $d \neq c_i$ verwerp.

Voor aanvaarden: Er is een run die voor elke knoop $v \neq s$, all knopen u in A_i heeft bezocht, en gechecked heeft dat (u, v) een boog is. T.t.z., dat $v \in A_{i+1}$ zit. Merk op: c_{i+1} is dan inderdaad $|A_{i+1}|$!

$M =$ “On input $\langle G, s, t \rangle$:

1. Let $c_0 = 1$. [[$A_0 = \{s\}$ has 1 node]]
2. For $i = 0$ to $m - 1$: [compute c_{i+1} from c_i]
3. Let $c_{i+1} = 1$. [c_{i+1} counts nodes in A_{i+1}]
4. For each node $v \neq s$ in G : [check if $v \in A_{i+1}$]
5. Let $d = 0$. [d re-counts A_i]
6. For each node u in G : [check if $u \in A_i$]
7. Nondeterministically either perform or skip these steps:
8. Nondeterministically follow a path of length at most i from s and *reject* if it doesn't end at u .
9. Increment d . [verified that $u \in A_i$]
10. If (u, v) is an edge of G , increment c_{i+1} and go to Stage 5 with the next v . [verified that $v \in A_{i+1}$]
11. If $d \neq c_i$, then *reject*. [check whether found all A_i]
12. Let $d = 0$. [c_m now known; d re-counts A_m]
13. For each node u in G : [check if $u \in A_m$]
14. Nondeterministically either perform or skip these steps:
15. Nondeterministically follow a path of length at most m from s and *reject* if it doesn't end at u .
16. If $u = t$, then *reject*. [found path from s to t]
17. Increment d . [verified that $u \in A_m$]
18. If $d \neq c_m$, then *reject*. [check that found all of A_m]
 Otherwise, *accept*.”

Wat bewaren we op de werk-tape?

- ▶ u en v (We hergebruiken space dus we moeten alleen de huidige knopen bijhouden)
- ▶ c_i en c_{i+1} (en dus niet alle c_i s)
- ▶ De tellers d en i
- ▶ Een pointer naar de knoop op paden die we kiezen.

Elk van deze vraagt LOGSPACE cellen.



Corollary

$NLOGSPACE \subseteq PTIME$.

Bewijs: We weten dat PATH in PTIME zit.

Daarenboven, elke taal L in NLOGSPACE is LOGSPACE-reduceerbaar naar PATH. Het is nu voldoende om in te zien dat een LOGSPACE-reductie ook een PTIME-reductie is.

Omdat een TM die $s(n)$ plaats gebruikt slechts $n2^{\mathcal{O}(s(n))}$ tijd kan gebruiken. Als $s(n)$ van orde $\mathcal{O}(\log(n))$ is, is dat dus in $\mathcal{O}(n^2)$ en dus PTIME $\square$

We hebben dus:

$$\begin{array}{ccccccccccccccc} LOGSPACE & \subseteq & NLOGSPACE & \subseteq & PTIME & \subseteq & NP & \subseteq & PSPACE & = & NPSpace & \subseteq & EXPTIME \\ || & & || & & || & & ? & & || & & || & & || \\ co-LOGSPACE & \subseteq & co-NLOGSPACE & \subseteq & co-PTIME & \subseteq & co-NP & \subseteq & co-PSPACE & = & co-NPSpace & \subseteq & co-EXPTIME \end{array}$$

We weten alleen dat $NLOGSPACE \not\subseteq PSPACE$, $PTIME \not\subseteq EXPTIME$, $NP \not\subseteq NEXPTIME$ (niet in deze cursus).

Om dit te kunnen definiëren hebben we een **zwakkere vorm** van reducties nodig (zwakker dan LOGSPACE).

Deze bestaan, zogenaamde **(first-order) FO-reducties**.

En UPATH is een LOGSPACE-compleet probleem hiervoor (undirected PATH probleem).

Het behandelen van FO-reducties valt buiten deze cursus.

- ▶ LOGSPACE en NLOGSPACE en voorbeelden (Sipser, 8.4) (in Sipser worden deze L en NL genoemd)
- ▶ NLOGSPACE-compleetheid en reducties (Sipser 8.5) (2SAT is niet in Sipser)
- ▶ Immerman-Szelepcsényi (Sipser 8.6)
- ▶ Verbanden tussen complexiteitsklassen.